

Backend Migration Roadmap

adVantage - from a local-first SQLite WASM app to an invite-only, multi-user deployment

Prepared 19 September 2026

The work is not the database swap. Porting SQLite to Postgres is about two days of mechanical change, because the query layer was already written as 43 functions that each take a database handle as their first argument. What costs real time is **tenancy** - there is currently no notion of a user anywhere in the data layer - and **rewiring the client**, which today refetches every mounted query after any write because local reads are free. Over a network they are not.

This plan is ordered so the riskiest, least reversible decisions are settled first, and so you never hold someone else's financial records without a tested way to back them up and give them back.

What stays, what goes

AREA	SIZE	DISPOSITION
packages/core	1,252 LOC	Unchanged. Money, dates, taxonomy, analytics, schedule expansion. No DOM, no database.
packages/db queries	43 functions	Ported. Signatures already server-shaped; add a tenant-scoped handle.
Drizzle schema	7 tables	Rewritten for Postgres with user_id from migration 0001. Mechanical.
Raw SQL	27 fragments	Reviewed. Only SUBSTR(date,1,7) is dialect-flavoured, and Postgres has it.
SQLite worker, bridge, runtime migrator	~600 LOC	Deleted. Replaced by a connection pool and Drizzle Kit.
apps/web	6,943 LOC	Kept. Only DbProvider and the data hooks are backend-shaped.

Before you write any code

Your records currently exist only in this browser's OPFS storage and nowhere else. Open Settings and download a backup today. It is also the file that makes you user number one on the new backend.

Phases at a glance

PHASE		EFFORT	RISK
0	Prepare and decide	half a day	-
1	Tenancy and auth spike	1-2 days	High - settles the rest
2	Port the query layer	3-4 days	Low
3	API layer	2 days	Low
4	Rewire the client	~1 week	High - most unknowns
5	Accounts, invites, deletion	2-3 days	Medium
6	Deploy and first invite	2-3 days	Medium
7	After the first users	ongoing	-

The plan, in order

PHASE 0 Prepare and decide

half a day

Nothing here is reversible-expensive, but each one blocks something later.

- [] Export your current data from Settings and confirm the JSON opens and looks complete.
- [] Create the Postgres project (Neon recommended - scale-to-zero, and branching gives every migration a throwaway database to test against).
- [] Create apps/api and a fresh tenant-first database package. Do not edit the existing one - the local-first app keeps working until parity.
- [] Decide the identity store now: Better Auth in the same Postgres, so user_id is a real foreign key and account deletion can cascade. Choose Clerk only if you would rather write no auth code and accept a second source of truth.

Done when: A backup file is on disk, an empty Postgres exists, and the monorepo has somewhere to put the new backend without disturbing the old one.

PHASE 1 Tenancy and auth spike

1-2 days · highest risk

The de-risking step. Do not port 43 queries and then discover the isolation model is wrong. Port four and prove it first.

- [] Schema with `user_id` on every table, in the very first migration. Retrofitting tenancy later is far harder than starting with it.
- [] Row Level Security policies on every table, with the request setting `SET LOCAL app.user_id` inside its transaction. App-level filtering becomes the fast path, not the only defence.
- [] A tenant-scoped database handle built once per request. Make the unscoped handle impossible to reach from route code, so a query cannot be written without a user.
- [] Better Auth wired with its Drizzle adapter, its tables inside your own migrations.
- [] Port four representative queries: one list read, one aggregate, one write, and `importBackup`.
- [] **Write the isolation test:** user A cannot read, update or delete any row of user B's - through the query layer and directly against the database.

The one that would actually hurt

`importBackup` currently deletes every row in every table before inserting. Unscoped in a multi-user world that is a cross-tenant wipe triggered by a file upload. Fix it in this phase, while it is still four queries and not forty-three.

Done when: The isolation test passes, and you know whether the remaining port is boring or interesting.

PHASE 2 Port the query layer

3-4 days

Mostly mechanical now that the shape is settled. The test suite is what makes it safe.

- [] Remaining 39 query functions moved behind the scoped handle.
- [] Four `integer({ mode: 'boolean' })` columns become real booleans.
- [] Re-read the 27 raw SQL fragments; 15 hardcode table names and deserve a second look.
- [] Port the 39 data-layer tests to run against Postgres. They encode real bugs - the unqualified-column fault that silently zeroed every balance, the outlook double-count guard - and they are your proof the port did not change behaviour.
- [] Keep money as integer centavos. If this becomes numeric or a float you will lose centavos and not notice for months.
- [] Keep client-generated UUIDs. They cost nothing and are what makes optimistic and offline writes possible later.

Done when: All 39 data-layer tests pass against Postgres, unchanged in intent.

PHASE 3 API layer

2 days

Thin by design. The query functions are already typed; the transport should not re-describe them.

- [] tRPC router over the ported functions - near-zero glue, and the React app gets end-to-end types without a second schema.
- [] Turn on request batching. This is what fixes the dashboard, which currently fires around twelve queries on load.
- [] Consider one endpoint per screen for the heavy views, rather than a seven-way `Promise.all` on the client.
- [] Error shape and status codes settled once, centrally.

Done when: Every screen's data can be fetched over HTTP, and the dashboard is one or two round trips rather than twelve.

PHASE 4 Rewire the client

~1 week · most unknowns

The other genuinely uncertain phase, and the one most likely to overrun. Go screen by screen.

- [] Replace `DbProvider` with an API client and session context.
- [] Replace the global-revision refetch in `useLiveQuery`. Bumping a counter and refetching everything is correct when reads cost 0.2 ms and a thundering herd when they cost 80 ms.
- [] Port one screen at a time: Transactions first - it is the most representative - then Dashboard, Budgets, Planned, Accounts and Categories, Settings.
- [] Loading and error states now matter on every screen. They did not before.
- [] Add optimistic writes where the lag is most obvious: adding a record, logging a planned payment.
- [] Leave packages/core alone. All 1,252 lines of it.

Done when: Every screen works against the API with no local database, and adding a record still feels immediate.

PHASE 5 Accounts, invites and deletion

2-3 days

What turns a single-user app into something you can hand to another person.

- [] Sign-up gated on an invite code. An invites table with a code, an optional email and a used_at timestamp is about thirty lines. Do not build more than that.
- [] Password reset or magic link, on a transactional email provider's free tier.
- [] Account deletion that genuinely deletes, via ON DELETE CASCADE from the user row.
- [] Per-user data export, reusing the existing backup format.
- [] First-run experience for a new account: the seed already handles this, but check it reads sensibly for someone who is not you - the default categories and two income sources are currently shaped around your situation.

Done when: You can invite a stranger, they can sign up, use it, export their data and delete their account without you touching the database.

PHASE 6 Deploy and first invite

2-3 days

Small scale, so skip the scale engineering. None of the items below are scale engineering.

- [] API on Fly.io or Railway; the web build stays static. Realistically under USD 15 a month.
- [] Migrations run in CI against a database branch before they touch production.
- [] **Take a backup, restore it into a scratch database, and confirm the data is there.** An untested backup is not a backup.
- [] HTTPS and secure, httpOnly session cookies.
- [] Scrub logging: no amounts, payees, or email addresses in request logs or error reports.
- [] Import your own backup as user number one, and check the figures match what the local app shows today.
- [] Send invite number one.

Done when: Someone other than you is using it, and you have restored a backup at least once.

PHASE 7 After the first users

ongoing

Deliberately thin. Add these when the first users exist, not before.

- [] Error tracking - Sentry's free tier is plenty at this size.
- [] An uptime check that tells you before your users do.
- [] Revisit the deferred items only when they start to hurt: rate limiting, admin tooling, read replicas, pooling.
- [] Resume feature work. The payday-based month is still unwired, and the debt payoff view is the obvious next thing given your installment schedule.

Done when: You are back to building the product rather than the platform.

Rules that hold across every phase

Money stays integer centavos	Every conversion already goes through one file. A port to numeric or float loses centavos quietly.
packages/core stays database-agnostic	It is the reason this is a port and not a rewrite. Nothing in it should learn about Postgres.
Tenancy before features	A missing WHERE clause leaks someone's salary and debts. The isolation test is written in phase 1, not phase 6.
Never hand-roll auth	No custom password hashing, no bespoke session tokens.
Build for the users you have	Invite-only means you can defer almost all scale work. Defer it.
Keep the old app running	Until the new one reaches parity, the local-first build is your fallback and your reference for correct numbers.

Go / no-go gates

Three points where it is worth stopping to check rather than carrying on by momentum.

AFTER	STOP UNLESS
Phase 1	The isolation test passes. If tenancy is awkward at four queries it will be unmanageable at forty-three.
Phase 4	Adding a record still feels immediate. If it does not, fix it here - it is the whole reason the app was pleasant to use.
Phase 6	You have restored a backup successfully. Do not send an invite before this.

What you are trading away

The app currently says, honestly, that your records never leave your browser. A server ends that, and takes instant reads and offline use with it. That is a real cost and worth naming rather than discovering later.

Two things soften it. Keep the export path prominent, so the trust story becomes "you can take your data out whenever you like" rather than nothing at all. And if offline turns out to matter more than you expect, local-first sync - ElectricSQL, PowerSync, Zero - keeps SQLite in the browser and syncs to the same Postgres. It is the youngest of the options and conflict resolution becomes your problem, but the door stays open as long as you keep client-generated UUIDs.

Where to start on Monday

Phase 1, and nothing else. Schema with user_id, RLS, Better Auth, four ported queries, and the test that proves user A cannot read user B. One or two days, and it tells you whether the remaining three weeks are routine or interesting.